Chapter 2 Elementary Programming



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Lecture 1 Review

- Create, compile, and run a Java program.
- Programming Errors

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Motivations

- How to solve practical problems programmatically

- Java primitive data types and related subjects, such as
  - variables,
  - constants,
  - data types,
  - operators,
  - expressions, and
  - input and output.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

Writing a program involves designing a strategy for solving the problem and then using a programming language to implement that strategy.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Algorithm

- An *algorithm* describes how a problem is solved by listing the actions that need to be taken and the order of their execution.

- Algorithms can help the programmer plan a program before writing it in a programming language.

- Algorithms can be described in natural languages or in *pseudocode* (natural language mixed with some programming code).



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Introducing Programming with an Example

- Computing the Area of a Circle
- The algorithm for calculating the area of a circle

1. Read in the circle's radius.
2. Compute the area using the following formula:

   area = radius * radius * pi;

3. Display the result.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

Every Java program begins with a class definition in which the keyword class is followed by the class name.

```
public class ComputeArea {
    // Details to be given later
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

Every Java program must have a **main** method where program execution begins.

```
public class ComputeArea {
   public static void main(String[] args){
      // Step 1: Read in radius
      // Step 2: Compute area
      // Step 3: Display the area
   }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

Step 1: Read in radius

- The program needs to read the radius entered by the user from the keyboard.
  - Reading the radius
  - Storing the radius in the program

- In order to store the radius, the program needs to declare a symbol called a *variable*.

- A variable represents a value stored in the computer's memory.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

- Choose descriptive names for variables:
  - **radius** for radius, and **area** for area
- To let the compiler know what **radius** and **area** are, specify their data types.
- That is the kind of data stored in a variable, whether integer, real number, or something else. This is known as *declaring variables.*



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

Primitive data types:

- Integers,
- Real numbers,
- Characters,
- Boolean types.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing a program

Use the keyword **double** to declare a floating-point variable.

```
public class ComputeArea {
   public static void main(String[] args){
        double radius;
        double area;
        // Step 1: Read in radius
        // Step 2: Compute area
        // Step 3: Display the area
    }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

ComputeArea.java

```
1 public class ComputeArea {
2   public static void main(String[] args) {
3     double radius;
4     double area;
5
6     // Assign a value to radius
7     radius = 20; // radius is now 20
8
9     // Compute area
10    area = radius * radius * 3.14159;
11
12    // Display the result
13    System.out.println("The area for the circle of radius " +
radius + " is " + area);
14   }
15 }
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Introducing Programming with an Example

Listing 2.1 Computing the Area of a Circle

This program computes the area of the circle.

ComputeArea

Note: Clicking the green button displays the source code with interactive animation. You can also run the code in a browser. Internet connection is needed for this button.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```

allocate memory for radius

radius no value



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

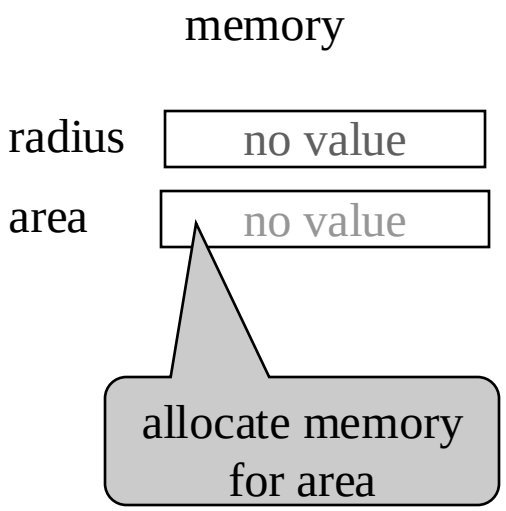
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

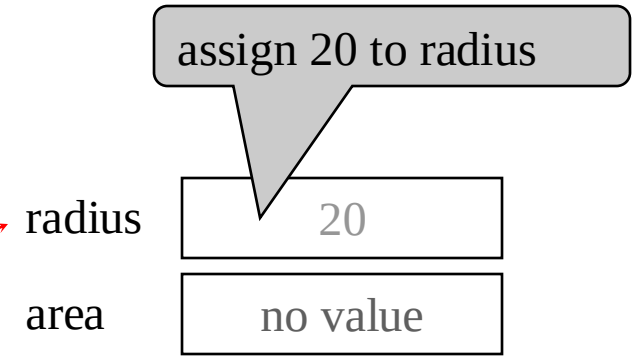
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

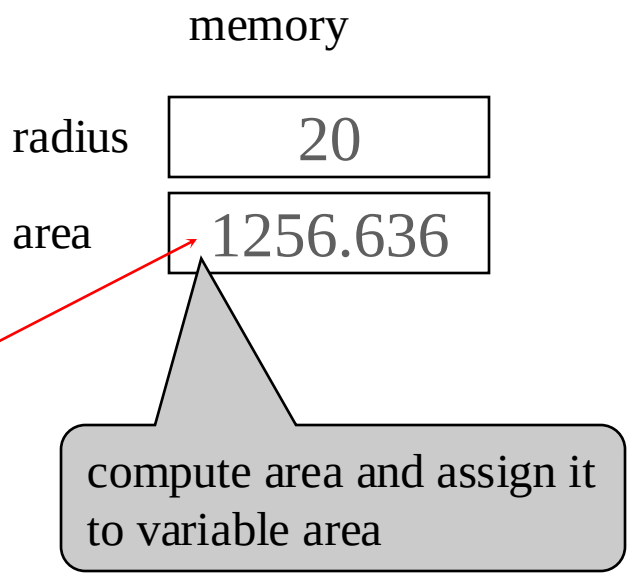
Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
public class ComputeArea {
  /** Main method */
  public static void main(String[] args) {
    double radius;
    double area;

    // Assign a radius
    radius = 20;

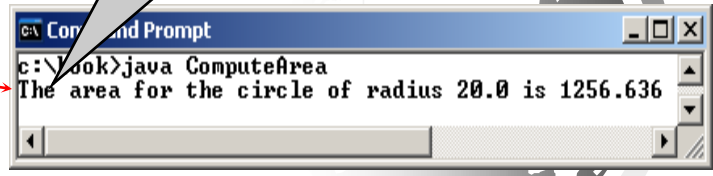
    // Compute area
    area = radius * radius * 3.14159;

    // Display results
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```

memory

radius	20
area	1256.636

print a message to the console



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Reading Input from the Console

- Reading input from the console enables the program to accept input from the user.
- Use the **Scanner** class for console input
- Java uses **System.out** to refer to the standard output device and **System.in** to the standard input device.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Reading Input from the Console

1. Create a Scanner object

```
Scanner input = new Scanner(System.in);
```

2. Use the method nextDouble() to obtain to a double value. For example,

```
System.out.print("Enter a double value: ");
Scanner input = new Scanner(System.in);
double d = input.nextDouble();
```



ComputeAreaWithConsoleInput

ComputeAverage

Copyright © 2022 Pearson Education Ltd. All rights reserved.

ComputeAreaWithConsoleInput.java

```
import java.util.Scanner; // Scanner is in the java.util package

public class ComputeAreaWithConsoleInput {
  public static void main(String[] args) {    // Create a Scanner object
    Scanner input = new Scanner(System.in);

    // Prompt the user to enter a radius
    System.out.print("Enter a number for radius: ");
    double radius = input.nextDouble();

    // Compute area
    double area = radius * radius * 3.14159;

    // Display result
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

ComputeAreaWithConsoleInput.java



```
Enter a number for radius:  2.5 Enter
The area for the circle of radius 2.5 is 19.6349375
```



```
Enter a number for radius:  23 Enter
The area for the circle of radius 23.0 is 1661.90111
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

ComputeAverage.java

```
import java.util.Scanner; // Scanner is in the java.util package

public class ComputeAverage {
  public static void main(String[] args) {
    // Create a Scanner object
    Scanner input = new Scanner(System.in);

    // Prompt the user to enter three numbers
    System.out.print("Enter three numbers: ");
    double number1 = input.nextDouble();
    double number2 = input.nextDouble();
    double number3 = input.nextDouble();

    // Compute average
    double average = (number1 + number2 + number3) / 3;

    // Display result
    System.out.println("The average of " + number1 + " " + number2 +
    " " + number3 + " is " + average);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

ComputeAverage.java

```
Enter three numbers: 1 2 3 Enter
The average of 1.0 2.0 3.0 is 2.0
```



enter input in one line

```
Enter three numbers: 10.5 Enter
11 Enter
11.5 Enter
The average of 10.5 11.0 11.5 is 11.0
```



enter input in multiple lines



Copyright © 2022 Pearson Education Ltd. All rights reserved.

import statement

- *specific import* specifies a single class in the import statement.

```
import java.util.Scanner;
```

- *wildcard import* imports all the classes in a package by using the asterisk

```
import java.util.*;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

print vs. println

- **println** moves to the beginning of the next line after displaying the string.

- **print** does not advance to the next line when completed.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Identifiers

- Identifiers are the names that identify the elements such as classes, methods, and variables in a program.
- An identifier is a sequence of characters that consist of letters, digits, underscores (_), and dollar signs ($).
- An identifier must start with a letter, an underscore (_), or a dollar sign ($). It cannot start with a digit.
- An identifier cannot be a reserved word.
  - (See Appendix A, "Java Keywords," for a list of reserved words).
- An identifier cannot be `true`, `false`, or `null`.
- An identifier can be of any length.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Variables

- Variables are used to represent values that may be changed in the program.

- To use a variable, you declare it by telling the compiler its name as well as what type of data it can store.

- The syntax for declaring a variable

```
datatype variableName;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Variables

```
// Compute the first area
radius = 1.0;
area = radius * radius * 3.14159;
System.out.println("The area is " +
  area + " for radius "+radius);

// Compute the second area
radius = 2.0;
area = radius * radius * 3.14159;
System.out.println("The area is " +
  area + " for radius "+radius);
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Declaring Variables

```
int x;          // Declare x to be an
                // integer variable;
double radius;  // Declare radius to
                // be a double variable;
char a;         // Declare a to be a
                // character variable;
int i, j, k;    //Declare i, j, and k as int
                //variables
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Assignment Statements

- An assignment statement designates a value for a variable.
- The equal sign (=) is used as the assignment operator .
- Syntax:

```
variable = expression;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Assignment Statements

```
x = 1;              // Assign 1 to x;

radius = 1.0;       // Assign 1.0 to radius;

a = 'A';            // Assign 'A' to a;



x = x + 1;          // The result of x+1 is
                       assigned to x

1 = x;              // Wrong
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

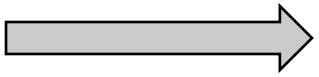
Declaring and Initializing in One Step

- **Syntax:**

```
datatype variable = expression;
```

- `int x = 1;` 

```
int x;
x = 1;
```

- `double d = 1.4;` 

```
double d;
d = 1.4;
```


Liang, Introduction to Java Programming and Data Structures, Twelfth Edition, Global Edition,
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Expression

An expression represents a computation involving values, variables, and operators that, taking them together, evaluates to a value.

```
int y = 1;                  // Assign 1 to variable y
double radius = 1.0; // Assign 1.0 to variable radius
int x = 5 * (3 / 2); // Assign the value of the expression to x
x = y + 1;                  // Assign the addition of y and 1 to x
double area = radius * radius * 3.14159; // Compute area
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Expression

- The following statement is true

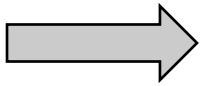
```
System.out.println(x = 1);
```



```
x = 1;
System.out.println(x);
```

- If a value is assigned to multiple variables, you can use this syntax:

```
i = j = k = 1;
```



```
k = 1;
j = k;
i = j;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Named Constants

- A named constant is an identifier that represents a permanent value.

- Syntax:

```
final datatype CONSTANTNAME = VALUE;
```

```
final double PI = 3.14159;
final int SIZE = 3;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

ComputeAreaWithConstant.java

```
public class ComputeAreaWithConstant {
  public static void main(String[] args) {
    final double PI = 3.14159; // Declare a constant

    // Create a Scanner object
    Scanner input = new Scanner(System.in);

    // Prompt the user to enter a radius
    System.out.print("Enter a number for radius: ");
    double radius = input.nextDouble();

    // Compute area
    double area = radius * radius * PI;

    // Display result
    System.out.println("The area for the circle of radius " +
      radius + " is " + area);
  }
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Naming Conventions

- Choose meaningful and descriptive names.
- Variables and method names:
  - Use lowercase. If the name consists of several words, concatenate all in one, use lowercase for the first word, and capitalize the first letter of each subsequent word in the name.
  - For example, the variables `radius` and `area`, and the method `computeArea`.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Naming Conventions, cont.

- Class names:
  - Capitalize the first letter of each word in the name. For example, the class name `ComputeArea`.

- Constants:
  - Capitalize all letters in constants, and use underscores to connect words. For example, the constant `PI and` MAX_VALUE



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Numerical Data Types

Name	Range	Storage Size
`byte`	-2^7 to $2^7 - 1$ (-128 to 127)	8-bit signed
`short`	-2^{15} to $2^{15} - 1$ (-32768 to 32767)	16-bit signed
`int`	-2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)	32-bit signed
`long`	-2^{63} to $2^{63} - 1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
`float`	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
`double`	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Reading Numbers from the Keyboard

```
Scanner input = new Scanner(System.in);
int value = input.nextInt();
```

Method	Description
`nextByte()`	reads an integer of the `byte` type.
`nextShort()`	reads an integer of the `short` type.
`nextInt()`	reads an integer of the `int` type.
`nextLong()`	reads an integer of the `long` type.
`nextFloat()`	reads a number of the `float` type.
`nextDouble()`	reads a number of the `double` type.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Division

- When both operands of a division are integers, the result of the division is the quotient and the fractional part is truncated.

  5 / 2 yields an integer **2**,

- To perform a floating-point division, one of the operands must be a floating-point number.

  5.0 / 2 yields a double value **2.5**.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Remainder Operator

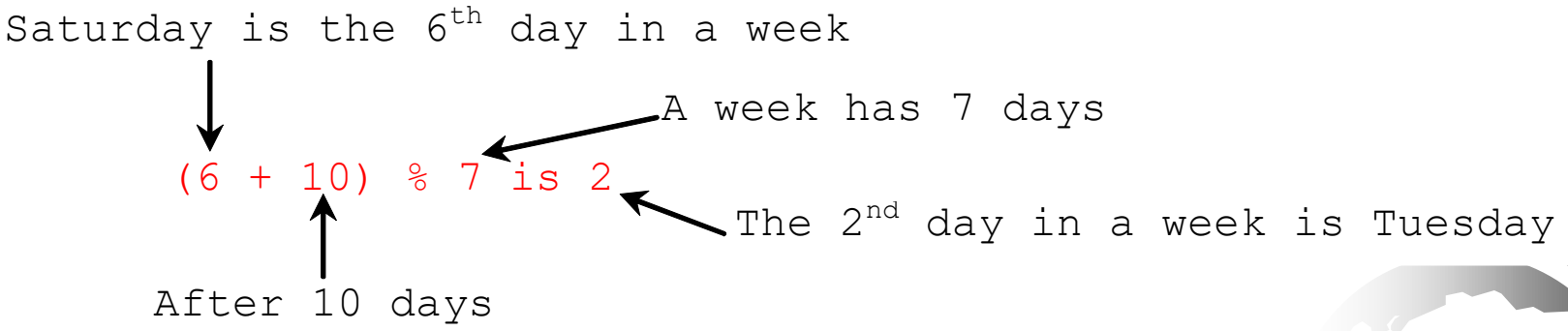
- The % operator yields the remainder after division.
- Remainder is very useful in programming.
- For example, an *even number* **% 2** is always **0** and an *odd number* **% 2** is always **1**.
  - So you can use this property to determine whether a number is even or odd.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Remainder Operator

Suppose today is Saturday and you and your friends are going to meet in 10 days. What day is in 10 days? You can find that day is Tuesday using the following expression:



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Displaying Time

Write a program that obtains minutes and remaining seconds from seconds.

DisplayTime



Copyright © 2022 Pearson Education Ltd. All rights reserved.

NOTE

Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy. For example,

System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);

displays 0.5000000000000001, not 0.5, and

System.out.println(1.0 - 0.9);

displays 0.09999999999999998, not 0.1. Integers are stored precisely. Therefore, calculations with integers yield a precise integer result.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exponent Operations

```
System.out.println(Math.pow(2, 3));
// Displays 8.0
System.out.println(Math.pow(4, 0.5));
// Displays 2.0
System.out.println(Math.pow(2.5, 2));
// Displays 6.25
System.out.println(Math.pow(2.5, -2));
// Displays 0.16
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Number Literals

A *literal* is a constant value that appears directly in the program. For example, 34, 1,000,000, and 5.0 are literals in the following statements:

```
int i = 34;

long x = 1000000;

double d = 5.0;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Integer Literals

- An integer literal can be assigned to an integer variable as long as it can fit into the variable.
- A compilation error would occur if the literal were too large for the variable to hold.
- For example, the statement **byte b = 1000** would cause a compilation error, because **1000** cannot be stored in a variable of the **byte** type.
  - *(Note that the range of byte value is from -128 to 127.)*



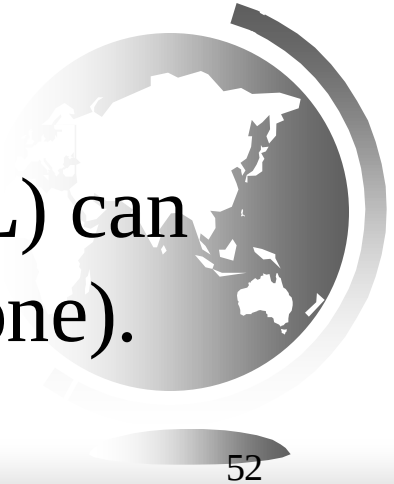
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Integer Literals

- An integer literal is assumed to be of the **int** type, whose value is between -2^{31} (-2147483648) to $2^{31}–1$ (2147483647).

- To denote an integer literal of the **long** type, append it with the letter **L** or **l**.

- **L** is preferred because **l** (lowercase **L**) can easily be confused with **1** (the digit one).



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Floating-Point Literals

- Floating-point literals are written with a decimal point.
- By default, a floating-point literal is treated as a **double** type value.
- For example, 5.0 is considered a **double** value, not a **float** value.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Floating-Point Literals

- You can make a number a **float** by appending the letter **f** or **F**, and make a number a **double** by appending the letter **d** or **D**.
- For example,
  - you can use **100.2f** or **100.2F** for a **float** number, and
  - **100.2d** or **100.2D** for a **double** number.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

double vs. float

The double type values are more accurate than the float type values. For example,

```
System.out.println("1.0 / 3.0 is " + 1.0 / 3.0);
```

displays `1.0 / 3.0 is 0.3333333333333333`

16 digits

```
System.out.println("1.0F / 3.0F is " + 1.0F / 3.0F);
```

displays `1.0F / 3.0F is 0.33333334`

7 digits



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Scientific Notation

- Floating-point literals can also be specified in scientific notation.
- For example,
  - 1.23456e+2, same as 1.23456e2, is equivalent to 123.456, and
  - 1.23456e-2 is equivalent to 0.0123456.
  - E (or e) represents an exponent and it can be either in lowercase or uppercase.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

JShell

JShell is a command line interactive tool introduced in Java 9. JShell enables you to type a single Java statement and get it executed to see the result right away without having to write a complete class. This feature is commonly known as REPL (Read-Evaluate-Print Loop), which evaluates expressions and executes statements as they are entered and shows the result immediately.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Arithmetic Expressions

$$\frac{3+4x}{5} - \frac{10(y-5)(a+b+c)}{x} + 9\left(\frac{4}{x} + \frac{9+x}{y}\right)$$

is translated to

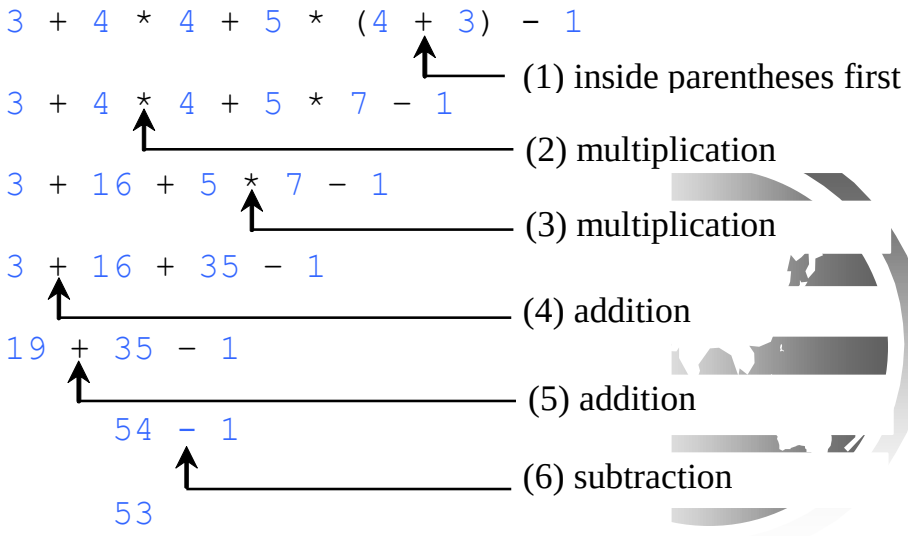
```
(3+4*x)/5 – 10*(y-5)*(a+b+c)/x + 9*(4/x + (9+x)/y)
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

How to Evaluate an Expression

Though Java has its own way to evaluate an expression behind the scene, the result of a Java expression and its corresponding arithmetic expression are the same. Therefore, you can safely apply the arithmetic rule for evaluating a Java expression.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Converting Temperatures

Write a program that converts a Fahrenheit degree to Celsius using the formula:

$$celsius = (\frac{5}{9})(fahrenheit - 32)$$

Note: you have to write
celsius = (5.0 / 9) * (fahrenheit – 32)

FahrenheitToCelsius

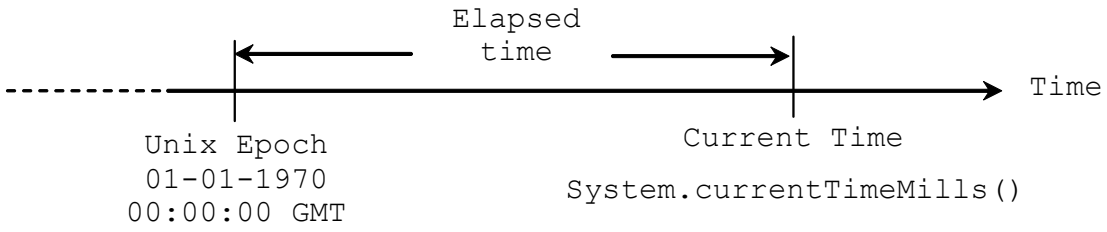


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Displaying Current Time

Write a program that displays current time in GMT in the format hour:minute:second such as 1:45:19.

The currentTimeMillis method in the System class returns the current time in milliseconds since the midnight, January 1, 1970 GMT. (1970 was the year when the Unix operating system was formally introduced.) You can use this method to obtain the current time, and then compute the current second, minute, and hour as follows.



ShowCurrentTime

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Augmented Assignment Operators



Operator	*Name*	*Example*	*Equivalent*
`+=`	Addition assignment	`i += 8`	`i = i + 8`
`-=`	Subtraction assignment	`i -= 8`	`i = i - 8`
`*=`	Multiplication assignment	`i *= 8`	`i = i * 8`
`/=`	Division assignment	`i /= 8`	`i = i / 8`
`%=`	Remainder assignment	`i %= 8`	`i = i % 8`



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Augmented Assignment Operators

- The augmented assignment operator is performed last after all the other operators in the expression are evaluated.
- For example,

```
x /= 4 + 5.5 * 1.5;
```

is same as

```
x = x / (4 + 5.5 * 1.5);
```



Caution!
There are no spaces in the augmented assignment operators. For example, + = should be +=.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example: Show the output of the following code

```
double a = 6.5;
a += a + 1;
System.out.println(a);
a = 6;
a /= 2;
System.out.println(a);
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Increment and Decrement Operators

Operator	*Name*	*Description*	*Example (assume i = 1)*
`++var`	preincrement	Increment `var` by `1`, and use the new `var` value in the statement	`int j = ++i;` `// j is 2, i is 2`
`var++`	postincrement	Increment `var` by `1`, but use the original `var` value in the statement	`int j = i++;` `// j is 1, i is 2`
`--var`	predecrement	Decrement `var` by `1`, and use the new `var` value in the statement	`int j = --i;` `// j is 0, i is 0`
`var--`	postdecrement	Decrement `var` by `1`, and use the original `var` value in the statement	`int j = i--;` `// j is 1, i is 0`



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Increment and Decrement Operators, cont.

```
int i = 10;
int newNum = 10 * i++;
```

Same effect as

```
int newNum = 10 * i;
i = i + 1;
```

```
int i = 10;
int newNum = 10 * (++i);
```

Same effect as

```
i = i + 1;
int newNum = 10 * i;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Increment and Decrement Operators, cont.

Using increment and decrement operators makes expressions short, but it also makes them complex and difficult to read.

Avoid using these operators in expressions that modify multiple variables, or the same variable for multiple times such as this: int k = ++i + i.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example: Show the output of the following code

```
int x = 3;
int y = 4;
int k = x * y++;
int l = x * ++y;
System.out.println("x " + x);
System.out.println("y " + y);
System.out.println("k " + k);
System.out.println("l " + l);
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Numeric Type Conversion

Consider the following statements:

```
byte i = 100;
long k = i * 3 + 4;
double d = i * 3.1 + k / 2;
```

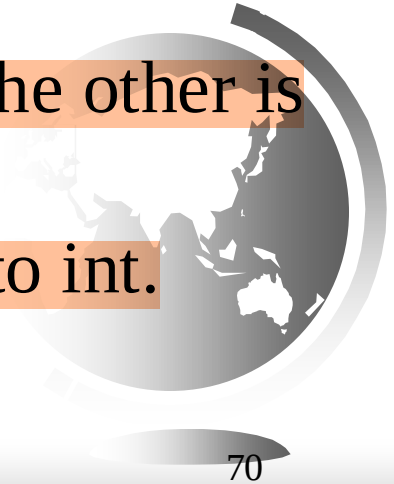


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Conversion Rules

When performing a binary operation involving two operands of different types, Java automatically converts the operand based on the following rules:

1. If one of the operands is double, the other is converted into double.
2. Otherwise, if one of the operands is float, the other is converted into float.
3. Otherwise, if one of the operands is long, the other is converted into long.
4. Otherwise, both operands are converted into int.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Type Casting

- *Casting* is an operation that converts a value of one data type into a value of another data type

Implicit casting

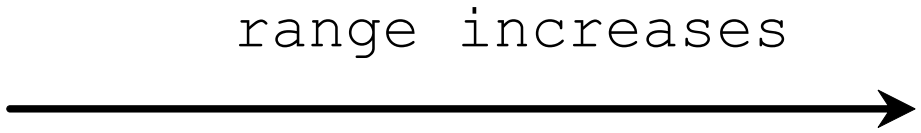
```
double d = 3;
```
(type widening)

Explicit casting

```
int i = (int)3.0;
```
(type narrowing)

```
int i = (int)3.9;
```
(Fraction part is truncated)

What is wrong? int x = 5 / 2.0;



byte, short, int, long, float, double



Copyright © 2022 Pearson Education Ltd. All rights reserved.

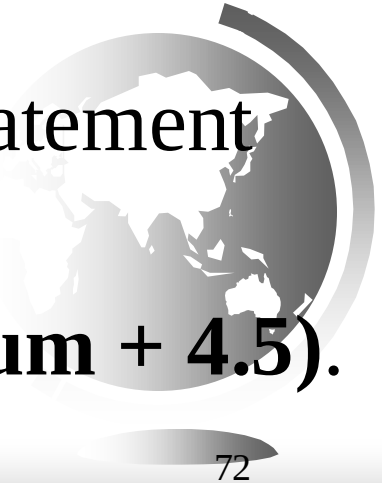
Casting in an Augmented Expression

In Java, an augmented expression of the form **x1 op= x2** is implemented as **x1 = (T)(x1 op x2)**, where **T** is the type for **x1**.

Therefore, the following code is correct.

```
int sum = 0;
sum += 4.5; // sum becomes 4 after this statement
```

sum += 4.5 is equivalent to **sum = (int)(sum + 4.5)**.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Keeping Two Digits After Decimal Points

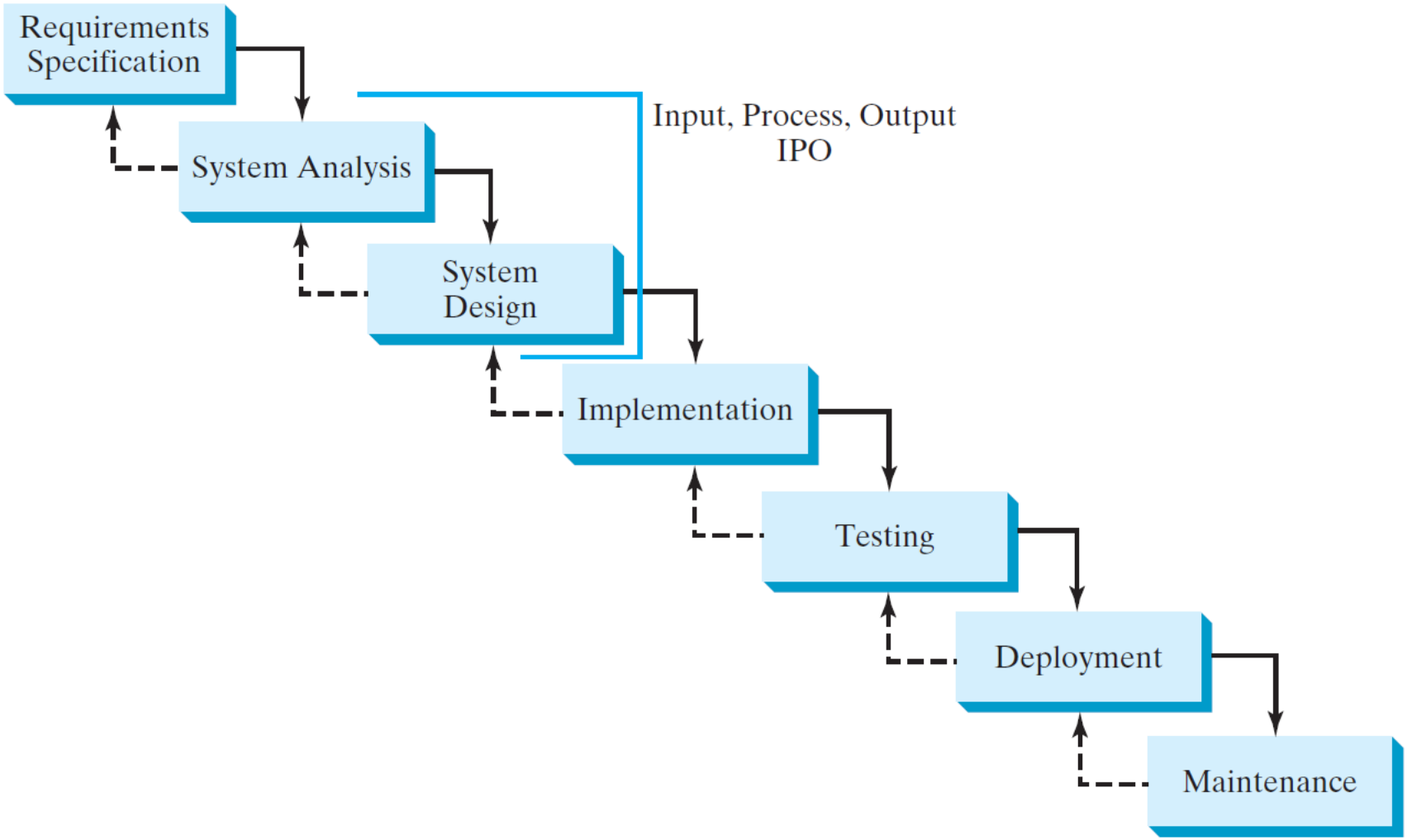
Write a program that displays the sales tax with two digits after the decimal point.

SalesTax



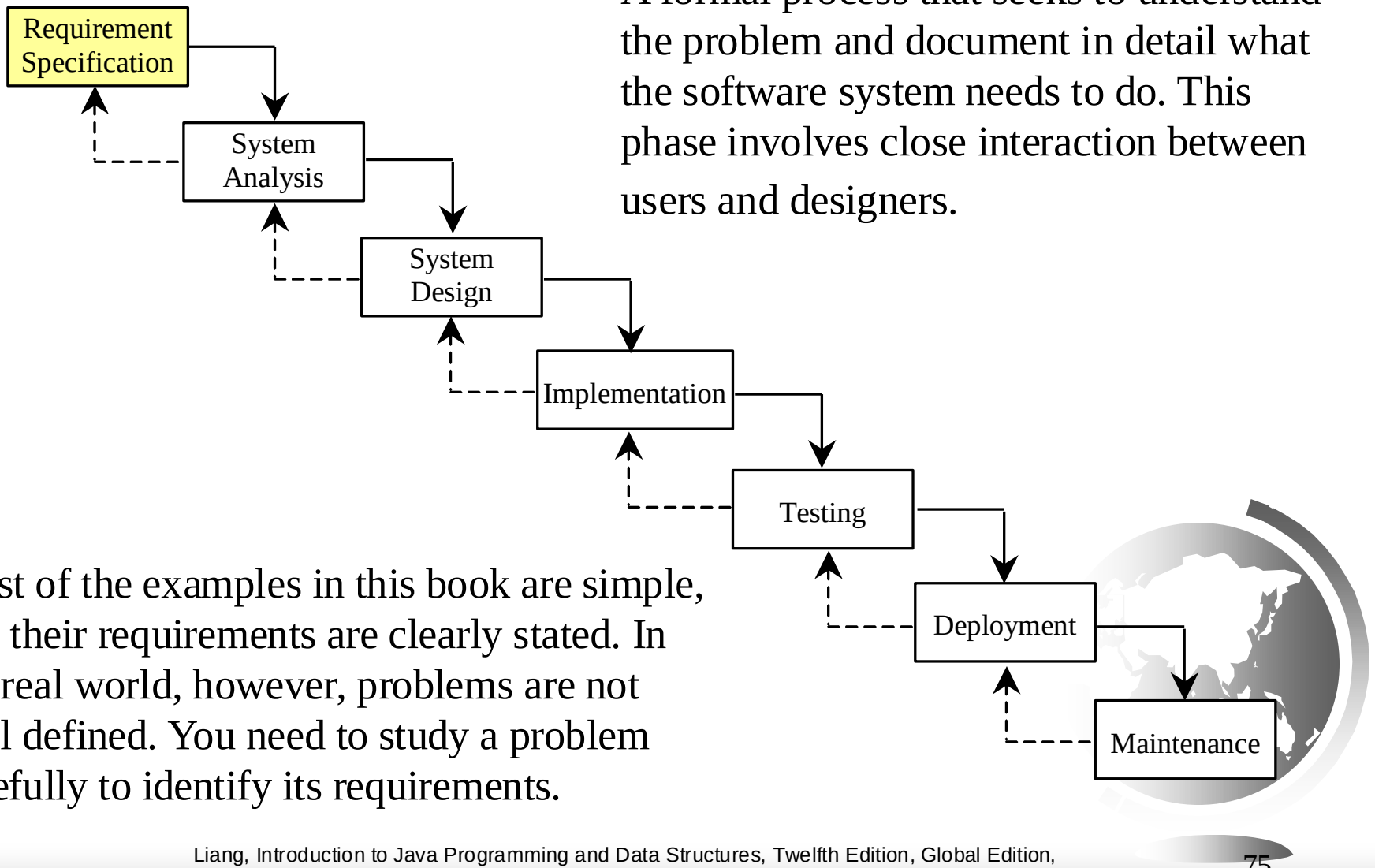
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Software Development Process



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Requirement Specification

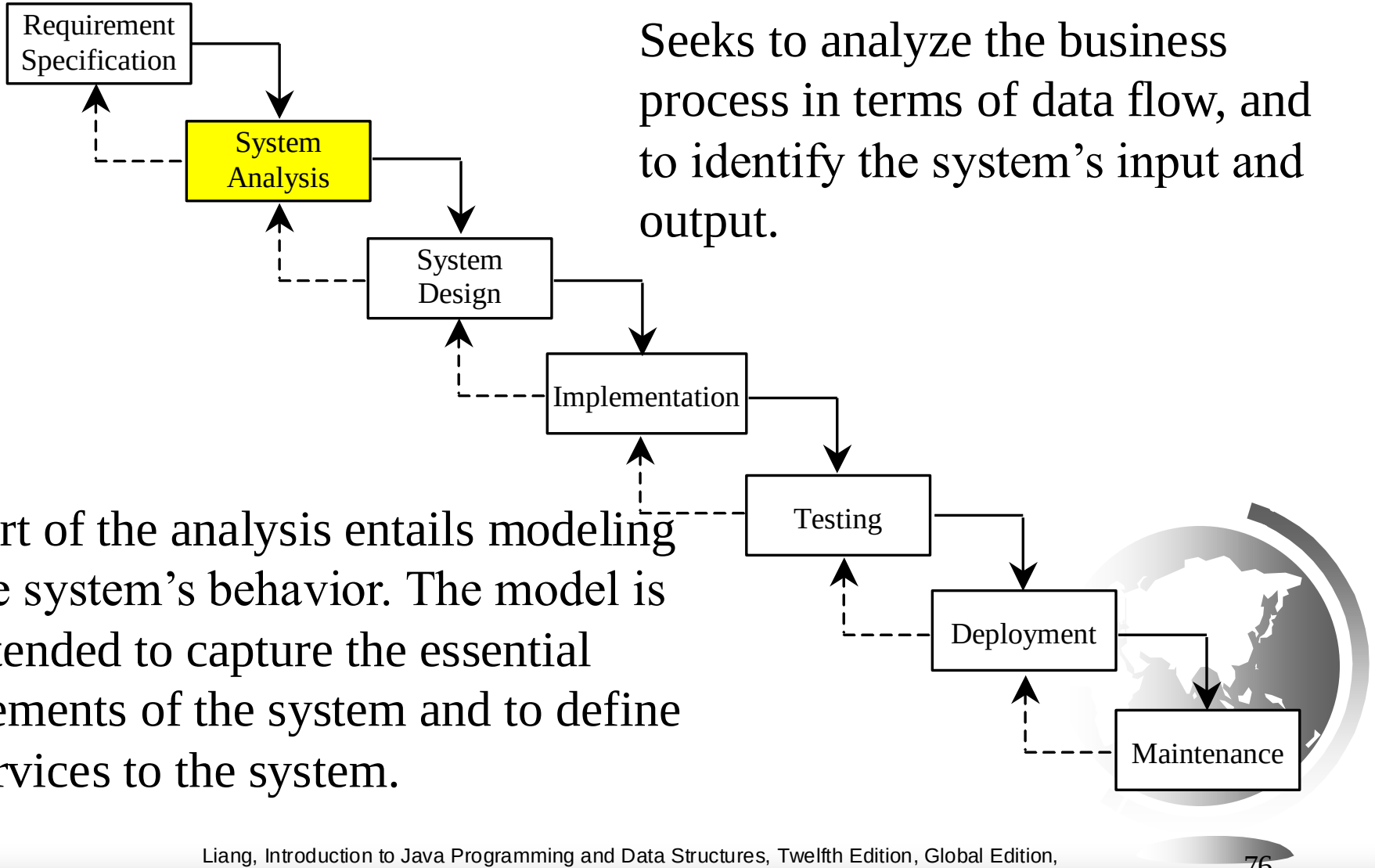


A formal process that seeks to understand the problem and document in detail what the software system needs to do. This phase involves close interaction between users and designers.

Most of the examples in this book are simple, and their requirements are clearly stated. In the real world, however, problems are not well defined. You need to study a problem carefully to identify its requirements.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

System Analysis

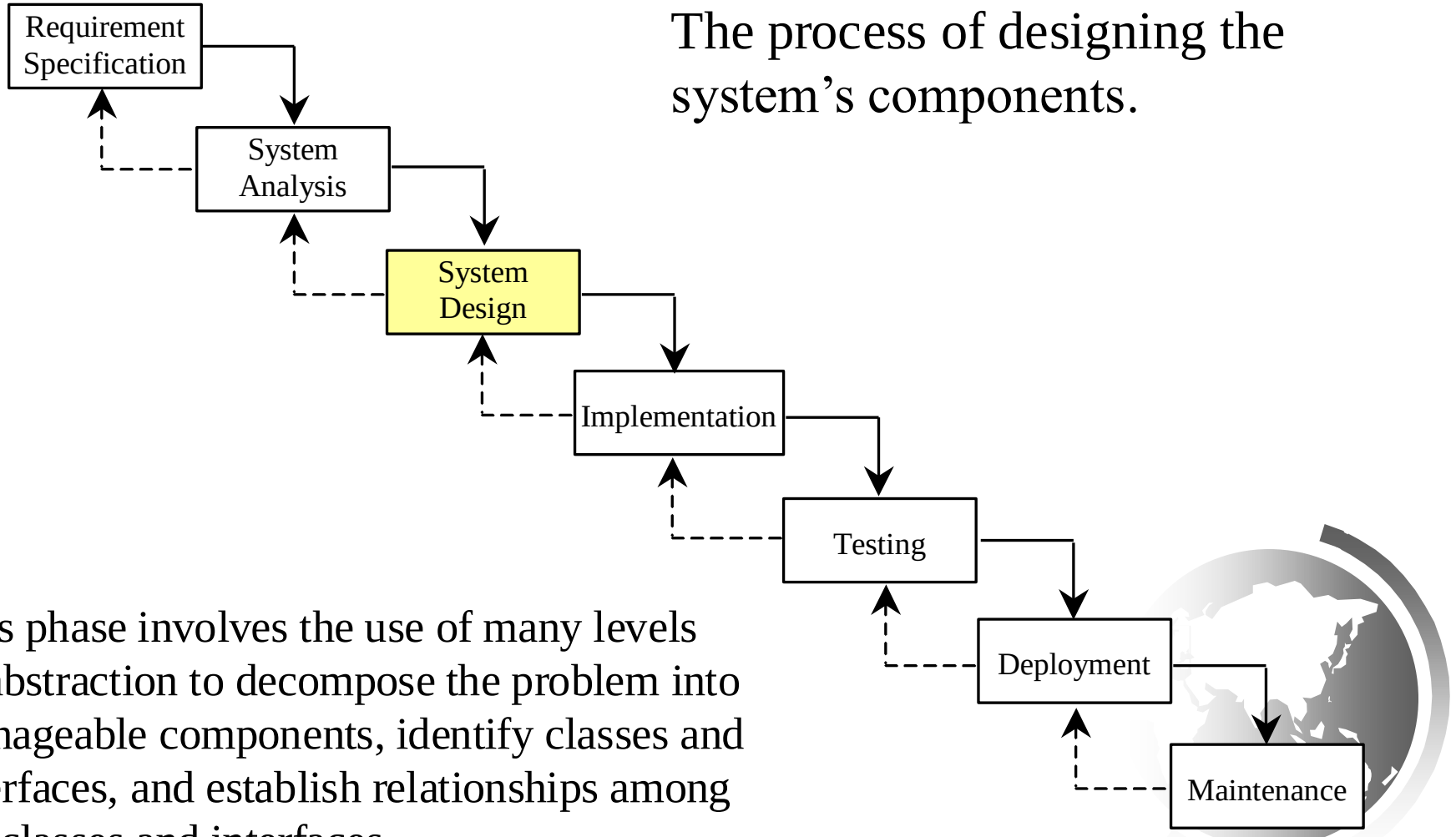


Seeks to analyze the business process in terms of data flow, and to identify the system's input and output.

Part of the analysis entails modeling the system's behavior. The model is intended to capture the essential elements of the system and to define services to the system.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

System Design

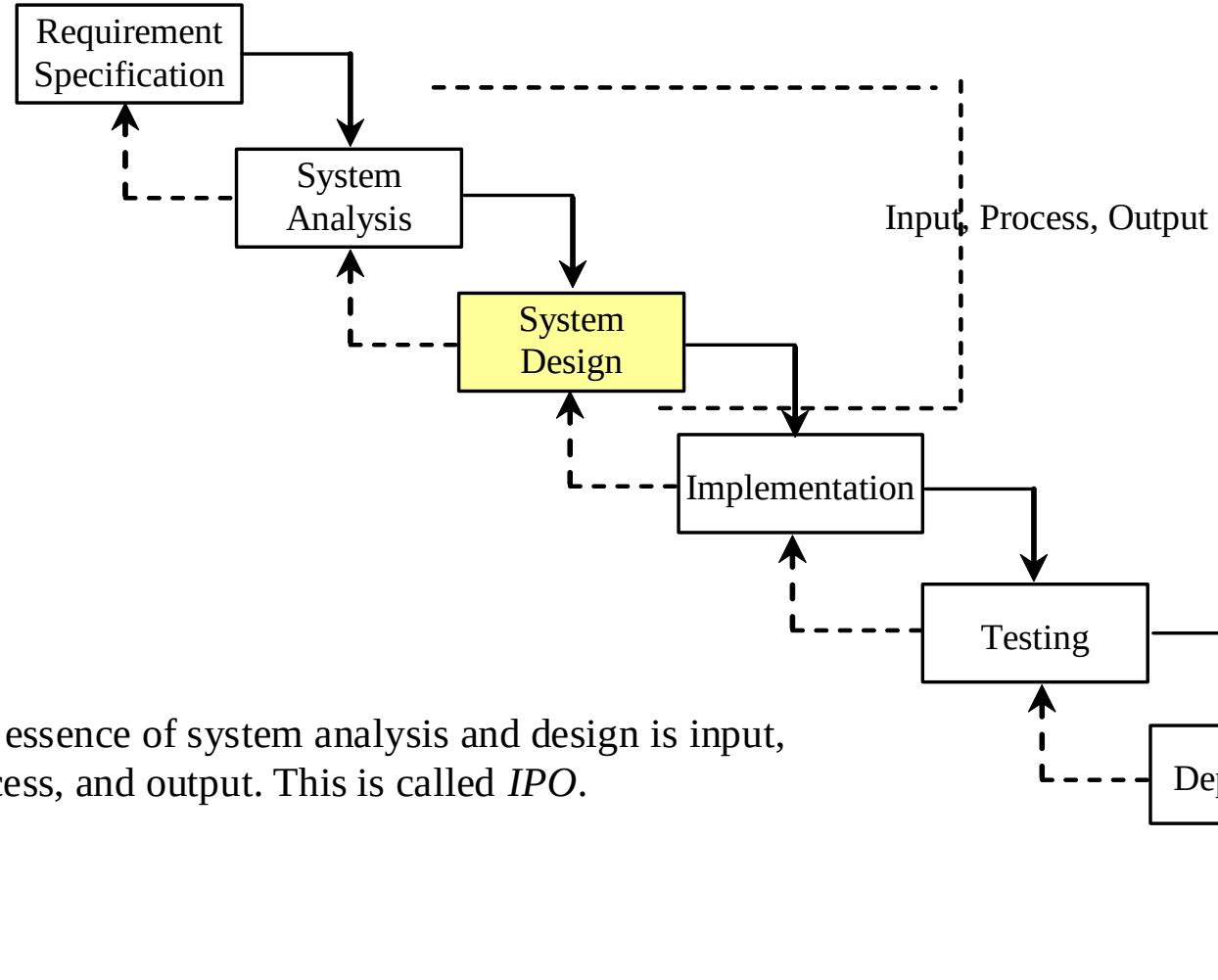


The process of designing the system's components.

This phase involves the use of many levels of abstraction to decompose the problem into manageable components, identify classes and interfaces, and establish relationships among the classes and interfaces.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

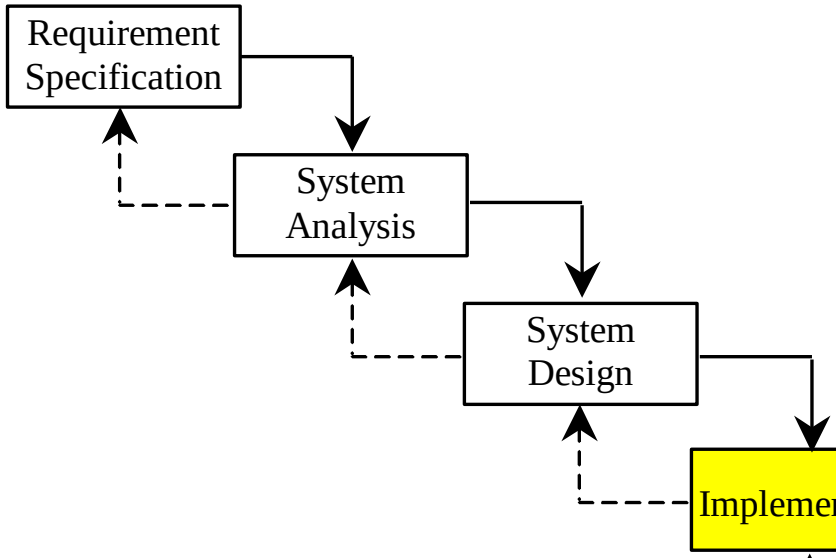
IPO



The essence of system analysis and design is input, process, and output. This is called *IPO*.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Implementation

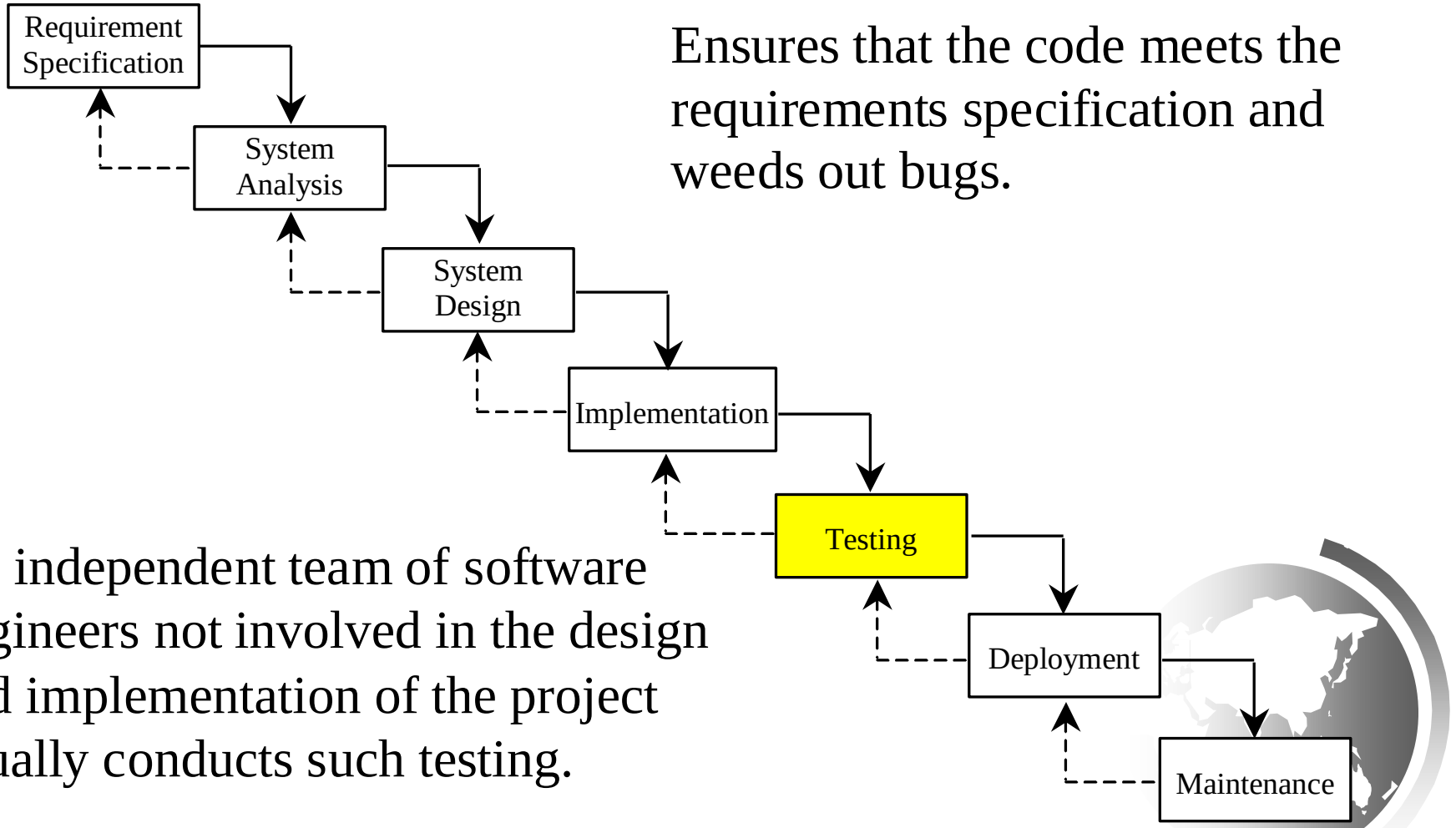


The process of translating the system design into programs. Separate programs are written for each component and put to work together.

This phase requires the use of a programming language like Java. The implementation involves coding, testing, and debugging.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Testing

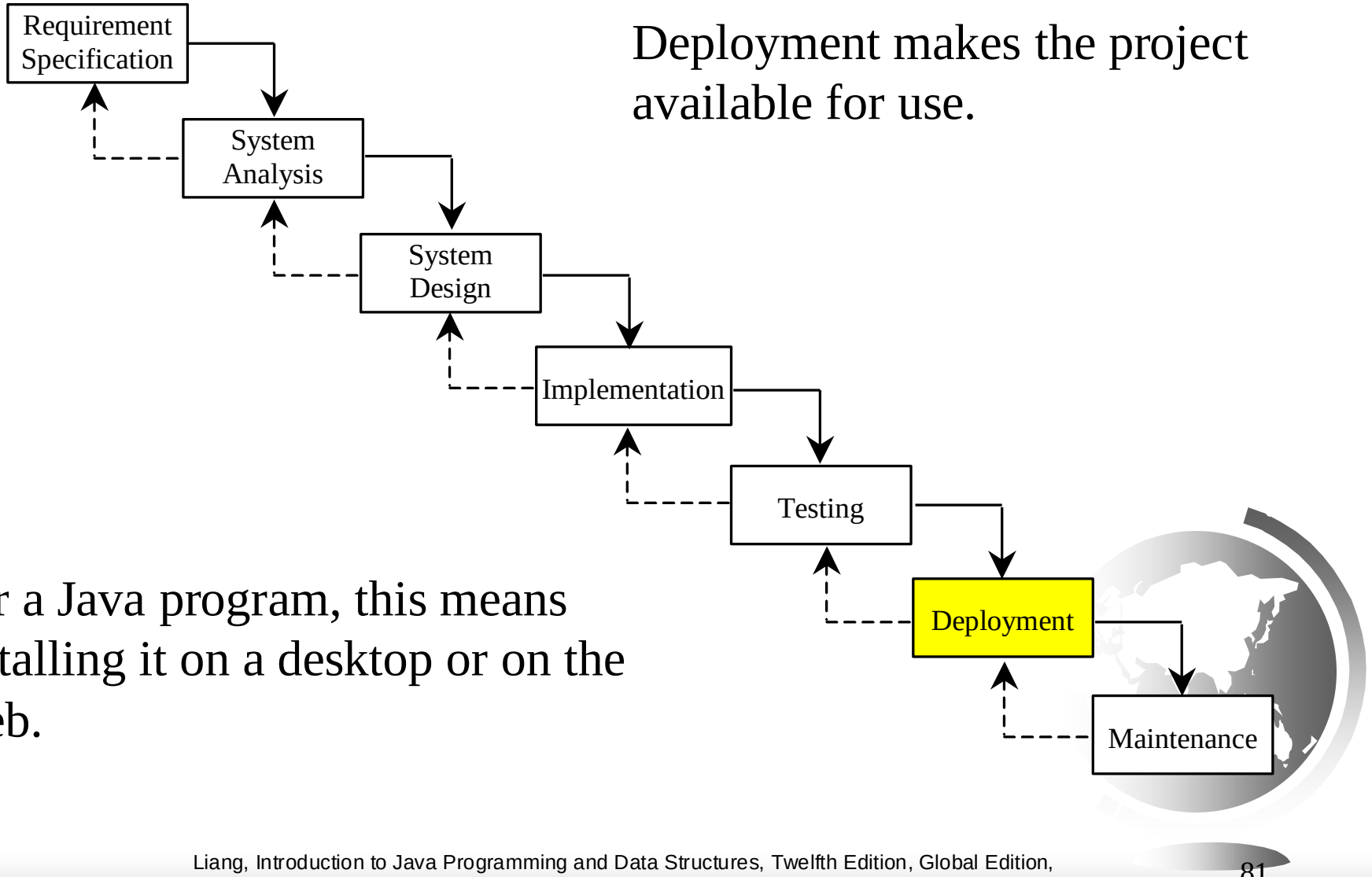


Ensures that the code meets the requirements specification and weeds out bugs.

An independent team of software engineers not involved in the design and implementation of the project usually conducts such testing.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Deployment

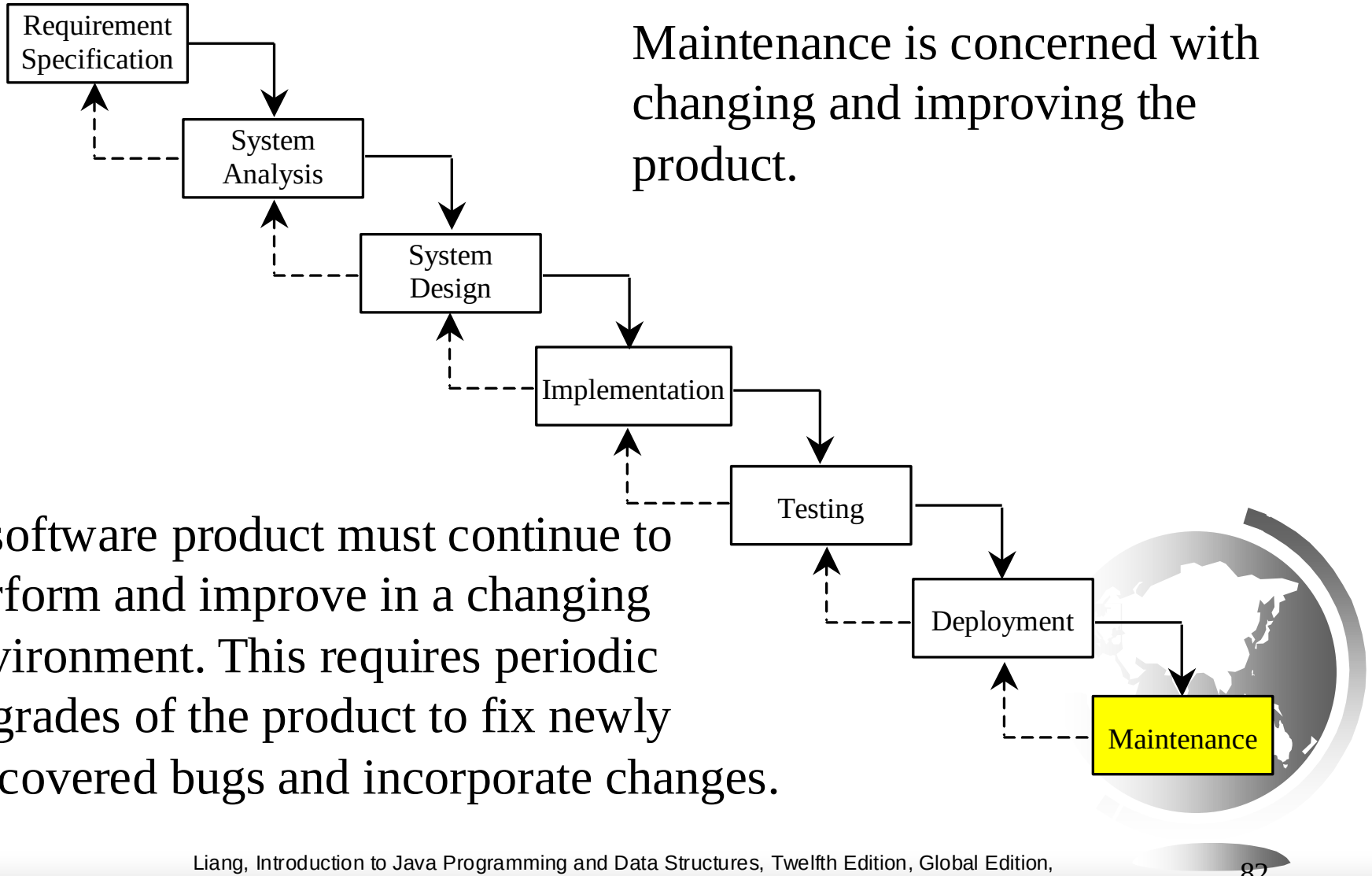


Deployment makes the project available for use.

For a Java program, this means installing it on a desktop or on the Web.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Maintenance



Maintenance is concerned with changing and improving the product.

A software product must continue to perform and improve in a changing environment. This requires periodic upgrades of the product to fix newly discovered bugs and incorporate changes.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Computing Loan Payments

This program lets the user enter the interest rate, number of years, and loan amount, and computes monthly payment and total payment.

$$monthlyPayment = \frac{loanAmount \times monthlyInterestRate}{1 - \frac{1}{(1 + monthlyInterestRate)^{numberOfYears \times 12}}}$$

ComputeLoan

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Errors and Pitfalls

- Common Error 1: Undeclared/Uninitialized Variables and Unused Variables
- Common Error 2: Integer Overflow
- Common Error 3: Round-off Errors
- Common Error 4: Unintended Integer Division
- Common Error 5: Redundant Input Objects

- Common Pitfall 1: Redundant Input Objects



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Error 1: Undeclared/Uninitialized Variables and Unused Variables

double interestRate = **0.05**;

double interest = interestrate * **45**;



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Error 2: Integer Overflow

```
int value = 2147483647 + 1;
// value will actually be -2147483648
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Error 3: Round-off Errors

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

- displays **0.5000000000000001**, not **0.5**, and

```
System.out.println(1.0 - 0.9);
```

- displays **0.09999999999999998**, not **0.1**



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Error 4: Unintended Integer Division

```java
int number1 = 1;
int number2 = 2;
double average = (number1 + number2) / 2;
System.out.println(average);
```

(a)

```java
int number1 = 1;
int number2 = 2;
double average = (number1 + number2) / 2.0;
System.out.println(average);
```

(b)



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Pitfall 1: Redundant Input Objects

```
Scanner input = new Scanner(System.in);
System.out.print("Enter an integer: ");
int v1 = input.nextInt();
```

```
Scanner input1 = new Scanner(System.in);
System.out.print("Enter a double value: ");
double v2 = input1.nextDouble();
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Common Pitfall 1: Redundant Input Objects (cont.)

```
Scanner input = new Scanner(System.in);
System.out.print("Enter an integer: ");
int v1 = input.nextInt();
System.out.print("Enter a double value: ");
double v2 = input.nextDouble();
```

GOOD CODE



Copyright © 2022 Pearson Education Ltd. All rights reserved.